# 05.1 — Change Management Policy

> **For:** Anyone making changes to Haritha Hospitals ERPNext (code, data, schema, infra). **Goal:** Minimize risk of bad changes reaching prod; ensure every change is reviewed, tested, and reversible. **Last updated:** 2026-08-29 **Pre-req reading:** 00.2 Architecture Overview, 04.1 Deployment, 04.4 Incident Response

---

## Purpose

Define **what counts as a change**, **who approves what**, and **what artifacts every change produces**. This is the policy layer above the runbooks (04.1 deployment, 04.2 daily ops, 04.4 incident response).

Without change management:

- Bad schema changes ship to prod with no review
- Master data migrations wipe existing rows
- Infra changes break without rollback path
- Audit trail is "I think we changed something last week?"

With change management:

- Every change has a record (date, author, risk, rollback)
- Reviews catch errors before they hit prod
- Rollback paths are pre-planned (not invented during incident)

---

## Change types

### 1. Code change

**What:** modifications to `haritha_hospital` app code (Python, JavaScript, hooks.py), custom scripts.

**Risk profile:** medium — bugs possible, but isolated to the app. Rollback = git revert.

**Examples:** - New method on `Employee` class - Modified `validate_approver` in Shift Request - New Jinja template for Print Format

**Approval path:** dev (solo) → QA (Venkat review) → prod (Venkat approval + smoke plan)

### 2. Data change

**What:** modifications to master data rows, mass updates, migrations.

**Risk profile:** high — can wipe data, break FK relationships, cause irreversible damage.

**Examples:** - `migrate_master_data.py` (16 DocTypes idempotent upserts) - `UPDATE tabEmployee SET status='Active'` mass update - Direct SQL DELETE on a table

**Approval path:** dev (solo) → QA (Venkat review) → prod (Venkat approval + dry-run output reviewed)

### 3. Schema change

**What:** new Custom Field, Property Setter, Print Format, Notification, Letter Head, or DocType.

**Risk profile:** medium — captured by fixtures, but schema drift can break reports / workflows.

**Examples:** - Add CF `Employee.haritha_employee_id` - Modify PS to make `Attendance.employee_name` readonly - New Print Format for Salary Slip

**Approval path:** dev (solo, must export fixtures immediately after) → QA (Venkat review) → prod (Venkat approval)

**Hard rule:** if a CF/PS/PF/N/LH is created in dev via UI, **`bench export-fixtures` must be run BEFORE promote** (Rule #9, Lesson #151).

### 4. Infra change

**What:** modifications to Docker containers, compose files, nginx config, SSL certs, backup scripts.

**Risk profile:** critical — no clean auto-rollback. Mistakes can take down entire env.

**Examples:** - Update `frappeclaw-compose.yml` (new service, image bump) - Modify nginx proxy config - Rotate SSH keys - Add new cron entry

**Approval path:** dev (solo for personal scripts) → ALL envs (Venkat explicit approval + rollback plan)

**Hard rule:** never `docker-compose down -v` on prod (Lesson #154: deletes named volumes).

---

## Approval requirements

| Env | Solo OK? | Approval needed? | Backup required? | Smoke test? |
|---|---|---|---|---|
| **dev** | ✅ | none | optional (last slot = baseline) | mandatory before promote |
| **qa** | ⚠ be careful | Venkat review | mandatory | mandatory |
| **prod** | ❌ never solo | **Venkat approval + smoke plan** | **mandatory (Lesson #79)** | **mandatory + Venkat ack** |

Single-operator model — there's no separate change advisory board. Venkat is the approver for QA and prod.

### Approval format

**Venkat expects (Telegram or commit comment):**

```
Promoting {commit} to {env} at {time}.
- Type: {code / data / schema / infra}
- Risk: {low / medium / high / critical}
- Backup: {timestamp + path}
- Rollback: {1-line summary}
- Smoke: {5 URLs to curl}
```

```
OK to proceed?
```

---

## Pre-flight checklist

Before ANY change to QA or prod, walk this:

```
[ ] 1.  Backup verified fresh (last successful slot)
        → ls -lt /home/vijay/backups/{env}/ | head -3
        → check SHA256 + offsite presence

[ ] 2.  If UI customization: exported as fixtures (Rule #9, Lesson #151)
        → apps/haritha_hospital/fixtures/*.json present
        → git committed + pushed

[ ] 3.  If new Python package or app install: backend restart planned
        → Lesson #153: docker restart erp-{env}-backend-1
        → expect 30-60s downtime per env

[ ] 4.  If data change: dry-run on dev first
        → show expected row counts (before/after)
        → show sample of what would change

[ ] 5.  If schema change: migration script idempotent
        → re-running produces same result
        → no destructive DELETE without WHERE clause
        → check `frappe.db.exists()` before `frappe.delete_doc()`

[ ] 6.  Tested on dev (8-phase test per Lesson #48)
      → smoke → vendor → schema → CRUD → workflow → integration → regression → docs

[ ] 7.  Rollback plan documented (see below)

[ ] 8.  Venkat notified before prod change
        → Telegram: "{change} to {env} at {time}. OK?"

[ ] 9.  MEMORY.md passwords verified fresh (Lesson #154)
        → docker exec erp-{env}-db-1 printenv MYSQL_ROOT_PASSWORD

[ ] 10. Off-hours if possible
        → prod changes preferred 22:00-04:00 IST
        → emergency: any time, notify Venkat first

[ ] 11. Change record started (see template below)
        → log date/time, author, change type, risk
```

---

## Change record

Every change to QA or prod gets a record. Minimal:

```
## Change Record — {date} {time} IST

**Author:** Venkat / subagent
**Env:** {dev / qa / prod}
**Type:** {code / data / schema / infra}
**Risk:** {low / medium / high / critical}
**Commit / files:** {git hash, file paths}
**Backup taken:** {path to backup bundle}
**Rollback steps:** {1-3 commands or "revert commit {hash}"}

### Change
{1-2 sentence description}

### Verification
{curl command + expected output}

### Rollback
{docker restart / restore / revert commit / etc.}

### Outcome
{filled in after change}
```

Where to store:

- **Per-env:** `docs/phase6/changes/{env}-YYYY-MM-DD.md` (one file per day)
- **Long-term:** appended to relevant runbook / LEARNINGS.md / MEMORY.md

---

## Rollback plan

Every change needs a pre-planned rollback. Don't invent this during an incident.

### Level 1 — Restart only (lightest)

```
docker restart erp-{env}-backend-1
```

**Use when:** new Python package, in-memory state issues, transient errors.

**Risk:** none (Lesson #154: restart preserves volumes).

### Level 2 — Revert code

```
docker exec erp-{env}-backend-1 bash -c '
  cd /home/vijay/frappeclaw/frappeclaw-data/workspace/projects/haritha-hospitals &&
  git checkout main && git reset --hard {last-good-commit} &&
  cd /home/frappe/frappe-bench/apps/haritha_hospital && git pull origin main'

docker restart erp-{env}-backend-1
```

**Use when:** bad commit, code bug.

**Risk:** low (git is safe). No data loss.

### Level 3 — Restore from backup

```
# Full procedure: see 04.3 Disaster Recovery §"Full prod rebuild"
```

**Use when:** data corruption, bad migration, lost data.

**Risk:** medium (15-30 min downtime, but reversible if backup is good).

#### Level 4 — Cold start

```
# Full procedure: see 04.3 Disaster Recovery §"Total loss"
```

**Use when:** total infra loss. Last resort.

**Risk:** high (4+ hour downtime, depends on cold storage availability).

---

## Test requirements

### Unit tests (code changes)

If adding/modifying Python code in `haritha_hospital`:

```
# Add to apps/haritha_hospital/haritha_hospital/tests/test_my_feature.py
import frappe
import unittest

class TestMyFeature(unittest.TestCase):
    def test_basic(self):
        # ...
        self.assertEqual(result, expected)
```

Run:

```
docker exec erp-dev-backend-1 bash -c '
  cd /home/frappe/frappe-bench &&
  bench --site pberpdev.duckdns.org run-tests --app haritha_hospital'
```

For minor changes: at least 1 happy-path test. For major changes: cover edge cases + error paths.

### Integration tests (env-level)

8-phase test playbook (Lesson #48):

1. **Smoke** — login + load home page (HTTP 200)
2. **Vendor tests** — `bench run-tests --app erpnext hrms haritha_hospital`
3. **Schema** — `tabError` Log empty after schema changes
4. **CRUD** — create / read / update / delete on each affected DocType
5. **Workflow** — submit / cancel / amend on workflow-enabled doctypes
6. **Integration** — cross-doctype flows (e.g., Attendance ← Shift Assignment ← Employee)
7. **Regression** — previously-working flows still work (Roster, Shift Type list)
8. **Docs** — fixtures exported + committed

### Smoke test (prod post-deploy)

Pre-define 5 URLs to curl + 3 pages to load in browser:

```
# After deploy to prod:
curl -sS -o /dev/null -w "%{http_code}\n" https://pberpPROD.duckdns.org/      # expect 302
curl -sS -X POST https://pberpPROD.duckdns.org/api/method/login \
```

```
  -H 'Content-Type: application/json' \
  -d '{"usr":"Administrator","pwd":"<from MEMORY>"}' | jq -r '.message'        # expect "S
curl -sS -b /tmp/cookies https://pberpPROD.duckdns.org/app/attendance \
  | grep -iE "500|error" | head -3                                             # expect e
curl -sS -b /tmp/cookies https://pberpPROD.duckdns.org/app/employee \
  | grep -iE "500|error" | head -3
curl -sS -b /tmp/cookies https://pberpPROD.duckdns.org/app/shift-type \
  | grep -iE "500|error" | head -3
```

Browser checks (manual, can't automate): - HR → Roster (loads, shows shifts) - HR → Attendance (loads, shows today's record) - HR → Shift Type (loads, custom fields visible if added)

---

## Communication

### Before change

**Prod change:** Telegram Venkat with: - Commit hash + 1-line description - Type + risk level - Backup timestamp - Rollback plan - ETA + smoke plan

### During change (long deploys)

Status update every 5 min if change > 10 min:

```
 pberpPROD deploy: 5 min in. Step 3/7 (migrate). No errors so far.
```

### After change

**All-clear:**

```
 pberpPROD deploy complete.
- Commit: {hash}
- Downtime: {X seconds/minutes}
- Smoke: all checks PASSED
- Rollback: not needed
```

**If any check fails:**

```
 pberpPROD deploy: smoke FAILED at step {N}.
- Symptom: {description}
- Action: {rollback Level X}
- Venkat: ack needed for next step
```

---

## Audit trail

Every change to prod leaves a trail:

1. **Git commit** (if code/schema) — `git log` shows who/when/what
2. **Backup bundle** (always) — `ls /home/vijay/backups/{env}/` shows pre-change snapshot
3. **Memory log** — `memory/YYYY-MM-DD.md` appended daily with all changes
4. **Change record** — `docs/phase6/changes/{env}-YYYY-MM-DD.md` (start this convention)
5. **Telegram history** — auto-archived by Telegram
6. **Supervisor logs** — `sites/{site}/logs/` for app-level events

For audit queries (“what changed to prod on 2026-08-29?”):

```
# Git history
git log --oneline --since="2026-08-29" --until="2026-08-30"

# Backups taken that day
ls /home/vijay/backups/prod/20260829*/

# Memory log
grep -iE "deploy|change|migrat|install" memory/2026-08-29.md
```

---

## Emergency changes (skip approval layers)

For SEV-1 / SEV-2 incidents where waiting for full approval = more damage:

```
1. Notify Venkat IMMEDIATELY (Telegram, no need to wait for response)
2. Execute safest fix (Level 1 restart before Level 4 restore)
3. Document as it happens (memory log, screenshots)
4. Get retroactive approval after the fact
5. Write post-mortem within 24h (see 05.2)
```

**Examples:** - Bad deploy causing HTTP 500 → `docker restart` immediately, notify Venkat during - Disk full on prod → clean up Docker, notify Venkat, restore if data lost - Security incident → patch + isolate, notify Venkat immediately

Retroactive approval is **not** a substitute for normal process. After an emergency change, Venkat reviews and either signs off or asks for additional safeguards.

---

## Common anti-patterns (DO NOT do)

| Anti-pattern | Why it’s bad | Do this instead |
|---|---|---|
| Edit code on prod container directly | No git history, no review, can’t rollback | Edit on dev, push, pull on prod |
| Run SQL UPDATE without WHERE clause | Wipes whole table | Always test on dev first, dry-run, narrow WHERE |
| `docker-compose down -v` on prod | Deletes named volumes (Lesson #154) | Always `docker restart` |
| `bench migrate` without backup | If migration fails mid-way, no recovery | Always backup first (Lesson #79) |
| Bulk re-tag `module IS NULL` | Catches Frappe stock Print Formats (Lesson #156) | Decision rule: skip creation < 2024 |
| Forget `docker restart` after install-app | New Python package invisible to gunicorn (Lesson #153) | Always restart after bench `install-app` |
| Skip smoke test on prod | Bug ships before anyone notices | Pre-define 5 curl URLs |
| Promote without backup | If something breaks, no recovery | Lesson #79 hardened backup mandatory |
| Trust MEMORY.md password literal | Passwords drift (Lesson #154) | Verify against container env first |

---

## References

- **Deployment runbook:** 04.1 Deployment Runbook
- **Daily ops:** 04.2 Daily Ops Runbook
- **Incident response:** 04.4 Incident Response Plan
- **Post-mortem template:** 05.2 Post-Mortem Template + Example
- **MEMORY.md standing rules:** “10 Industry Best-Practice Rules” + “Pre-Flight Checklist”
- **Lessons cited:** #5, #9, #44, #45, #46, #48, #72, #79, #98, #151, #152, #153, #154, #155, #156, #157

*Process > heroics. Approval > speed. Rollback > regret.*